SeedingMinds

Assessment Name

Java_S2_271225

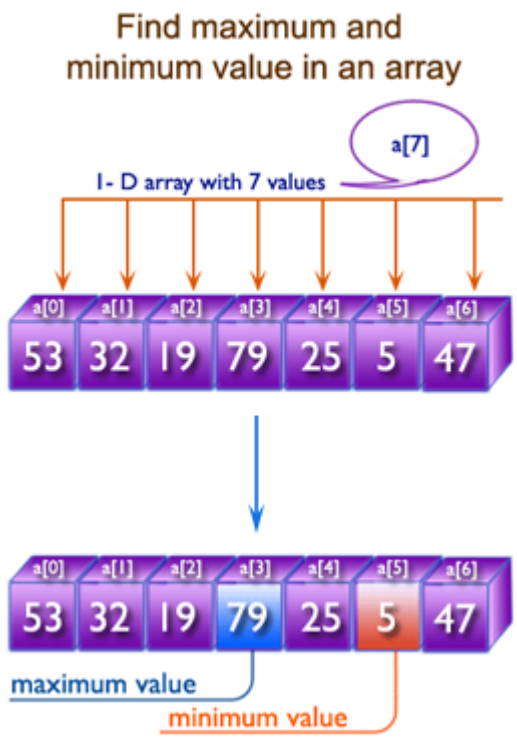
Module: Coding

Question 1:

Marks: +10 -0

find the maximum and minimum value of an array.

Write a Java program to find the maximum and minimum value of an array.



Constraints:

N/A

Example:

Input:

25, 14, 56, 15, 36, 56, 77, 18, 29, 49

Output:

77

14

Explanation:

-

Public Test Cases:

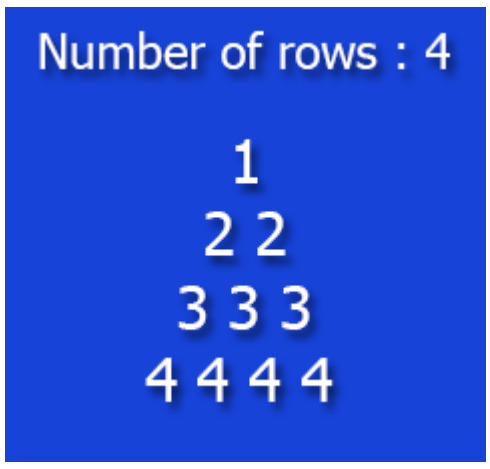
#	INPUT	EXPECTED OUTPUT
1	25, 14, 56, 15, 36, 56, 77, 18, 29, 49	77 14

Question 2:

Marks: +10 -0

print such pattern like a pyramid with a number which will repeat the number in the same row.

Write a program in Java to print such pattern like a pyramid with a number which will repeat the number in the same row.



Constraints:

N/A

Example:

Input:

4

Output:

```
  1
 2 2
3 3 3
4 4 4 4
```

Explanation:

-

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	4	1 2 2 3 3 3 4 4 4 4

Question 3:

Marks: +10 -0

Library Late Fee Calculation

A library charges a fine for every book returned late. For first 5 days the fine is 50 paise, for 6-10 days fine is one rupee and above 10 days fine is 5 rupees. If you return the book after 30 days your membership will be cancelled. Write a program to accept the number of days the member is late to return the book and display the fine or the appropriate message.

Note: Round of the result to 2 decimal points.

Constraints:

Input Format:

Line1: Integer value (represents the number of days overdue)

Output Format:

Line 1: Floating point number rounded off to 2 decimal points (represents the fine amount)

(or) -1 --> If the membership has been cancelled

Example:

Input:

4

Output:

2.00

Explanation:

Explanation:

The number of days the book was overdue - 4 days

For first 5 days the fine is 50 paise. So for 4 days the amount is Rs.2 and that is displayed as the output.

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	4	2.00
2	8	8.00

Question 4:

Marks: +10 -0

Email Format Validator for User Signup System

In today's digital world, almost every online platform (like **LinkedIn**, **GitHub**, or **EdTech portals**) requires users to sign up using their **email address**.
As part of the **backend input validation module** for a signup system, you are tasked with developing a **basic email validator** in Java.

The system should verify if the entered email address follows these **validation rules**:

1. The email must contain **exactly one '@' symbol**.
2. The email must contain **at least one '.' symbol after** the '@' symbol.
3. The email must **start with a lowercase letter (a–z)**.
4. The email must **end with one of the valid domain extensions**:
   - `.com`
   - `.in`
   - `.org`
   - `.edu`
5. The email must **not contain any spaces**.

If all these rules are met, the program should print **"Valid Email"**, otherwise **"Invalid Email"**.

Constraints:

- Email length ≤ 100 characters
- Allowed characters: lowercase letters, digits, `@` , `.` , `_`
- No usage of library functions like `strlen()` , `strchr()` , or regex.
- Use only **arrays, strings, conditional and control statements**.
- No functions, pointers, structs, or data structures.

Example:

Sample Input:

john_doe25@gmail.com

Sample Output:

Valid Email

Explanation:

NA

Public Test Cases:

#	INPUT	EXPECTED OUTPUT
1	`JohnDoe@domain.org`	`Invalid Email`